Meadow House Repairs

A storm damages the garden fence at fictional Meadow House. Staff need a clear way to report the fault, see repair progress and know when the work is complete. Other faults must remain separate, and earlier updates must stay visible.

Your challenge

Build a small working app that takes a repair from its first report to completion. Your team chooses the design and implementation. The published tests below define the behaviour we will assess.

1. Report a fault

Enter a description, reporter name and priority (**Normal** or **Urgent**). The home is always **Meadow House**. Generate the job ID and date automatically. Save a new report as **New**. Empty or spaces-only descriptions and reporter names must be rejected.

2. Track the repair

List and open multiple jobs independently. Let a user record progress and change the status between **New**, **In Progress** and **Completed**. Show each job's latest note and dated/timed status history.

3. Finish and retain the record

Require a non-empty completion note before saving **Completed**. A rejected completion must preserve the earlier saved state. Keep previous updates readable. A browser refresh must retain saved jobs, priorities, notes, statuses and history. Display typed content as plain text.

What we will mark

80 functional marks: ten published cases, worth 8 each. A case earns 8 when its complete expected behaviour is demonstrated, otherwise 0. **20 UI marks:** the twenty observable checks on page 4, worth 1 each.

Accounts, staff assignment, scheduled dates, photos, costs, deployment and an AI service inside the app are not required. A simple local app is sufficient. Your choice of framework or AI tool earns no extra marks.

Optional approach: plan with Claude, then build with Cursor, Antigravity, Codex or another university-permitted tool. You may use any permitted approach and must understand and test your work. Free tool limits vary.

Functional tests M01–M05

Each case is worth **8 marks**. Use the same examples and a fresh starting state for every team. A judging-tool failure requires review; it is not an automatic zero.

M01 / Create a valid report

Do: Enter **Garden fence damaged**, reporter **Alex Morgan** and **Urgent**, then submit.

Pass: A distinct job appears as **New**, with Meadow House, an automatically generated ID/date, and the submitted description, reporter and priority.

M02 / Reject a missing description

Do: Try an otherwise valid report with an empty description, then one containing only spaces.

Pass: Neither attempt creates a job. A useful error explains what needs fixing.

M03 / Reject a missing reporter

Do: Try an otherwise valid report with an empty reporter name, then one containing only spaces.

Pass: Neither attempt creates a job. A useful error explains what needs fixing.

M04 / Keep each priority

Do: Create one **Normal** report and one **Urgent** report, then inspect both.

Pass: Each job retains its own selected priority. Urgent is shown in words, so a user does not need colour to identify it.

M05 / Keep separate jobs independent

Do: Create fence and kitchen-light jobs. Open and update only the fence job.

Pass: Both jobs remain present. The fence job changes as requested; the kitchen-light job stays unchanged.

Functional tests M06–M10

Each case is worth **8 marks**. Use the same examples and a fresh starting state for every team. A judging-tool failure requires review; it is not an automatic zero.

M06 / Record progress

Do: Open an existing job, select **In Progress**, enter a progress note and save.

Pass: That job shows the saved status and latest note, plus a dated/timed history entry. The earlier record remains visible.

M07 / Require a completion explanation

Do: Try saving **Completed** without a completion note, then with a spaces-only note.

Pass: Both attempts are blocked. A clear message is shown, and the job's earlier saved status, notes and history remain intact.

M08 / Complete the repair with its history

Do: After recording progress, save **Completed** with the note **Fence repaired and checked**.

Pass: Completed and the completion note appear together. The earlier New and In Progress history remains visible.

M09 / Retain saved work on refresh

Do: Create two jobs and complete one. Refresh the browser, then inspect both jobs.

Pass: Both jobs remain, including their own priorities, saved notes, statuses, completion explanation and history.

M10 / Show typed content safely

Do: Enter the literal text **check gate** in a report and a saved note, then view them.

Pass: The full typed characters are displayed as text. They are not interpreted as HTML, formatted as markup or executed.

The 20 UI marks

Each fully demonstrated check earns **1 mark**. All five checks must pass for **5/5** in a category. Record an observation; an unobserved check remains pending.

Workflow / 5

1. Find the main reporting action within 10 seconds without a hint.
2. Find and open a new job within 30 seconds.
3. Find the progress and completion controls within 30 seconds.
4. Identify the saved status and latest note within 20 seconds.
5. Finish all three core features without inspecting source code or receiving coaching.

Readability / 5

1. At 1440 x 900, core screens have no clipping, overlapping content or page-level horizontal scrolling.
2. At 390 x 844, required controls and text remain reachable without page-level horizontal scrolling.
3. Main text, labels and actions are at least 14px; secondary instructions and history are at least 12px.
4. Actions and states use consistent names. Interactive controls are distinguishable from static text.
5. A 200-character note wraps or expands so all of it can be read.

Feedback / 5

1. Submitting a valid report clearly confirms that the job saved.
2. Invalid actions show understandable messages and preserve other entered content.
3. Saving progress clearly shows the saved result.
4. Urgent has a written label, rather than relying on colour alone.
5. Completed appears together with its completion explanation.

Accessibility / 5

1. Every control has a programmatic accessible name. A placeholder alone is insufficient.
2. All core tasks work by keyboard with no keyboard trap.
3. Keyboard focus is visible on every main control.
4. Essential text contrast is at least 4.5:1; large text is at least 3:1 (24px regular or 18.66px bold).
5. Errors and result changes use associated descriptions, live status or deliberate focus on a named result.

Timing excludes agreed accessibility adjustments and judge/setup delays. Use the same examples and screen sizes for both sessions. Full marks may fairly go to every team that meets every check.